To further secure your application: - Always use `{{ }}` unless raw HTML is absolutely necessary. - Sanitize user input using libraries like HTMLPurifier when displaying user-generated content.

16.4 SQL Injection Prevention

Laravel uses prepared statements and parameter binding by default, making SQL injection attacks highly unlikely when using Eloquent or the Query Builder:

```
// Safe with Eloquent
User::where('email', $email)->first();

// Safe with Query Builder
DB::table('users')->where('email', $email)->get();
```

Avoid using raw SQL queries unless absolutely necessary. If you must use `DB::raw()`, ensure all inputs are sanitized properly.

16.5 CSRF Protection

Cross-Site Request Forgery (CSRF) tricks users into performing actions without their consent. Laravel automatically protects forms with CSRF tokens:

```
<form method="POST" action="/submit">
    @csrf
    <!-- form fields -->
</form>
```

For API routes, CSRF protection is typically disabled by default since APIs are stateless. However, you should still: - Authenticate requests via tokens (e.g., JWT, Sanctum). - Use HTTPS to protect token transmission.

16.6 Authentication and Authorization Best Practices

Laravel provides robust authentication and authorization systems. Use Laravel Breeze, Jetstream, or Fortify to implement secure login systems